

Tract-AI Repo Audit

Reviewer: Claude Code · Date: 2026-05-17 · Branch: `main` @ 98e9b88

Scope: ~4,100 LOC across 9 source files. Backend = Node/Express 5 (CommonJS); Frontend = vanilla JS + Vite, deployed via Netlify. Supabase for persistence (optional).

1. Repo setup — 6/10

Area	Notes
Structure	Two-package: root for backend deps, <code>frontend/</code> for Vite. Reasonable.
Env files	Mismatch. <code>backend/server.js:2</code> and <code>backend/supabaseClient.js:4</code> read <code>path.join(__dirname, "..", ".env")</code> — i.e. project root — but the file actually lives in <code>backend/.env</code> . Without a symlink, the API loads zero env vars and Supabase <code>createClient(undefined, ...)</code> throws. Either move the file or fix the path.
Migrations	<code>backend/supabase-migrations/</code> contains only <code>alter table</code> files. No base create table public.scans — fresh Supabase projects 500 on every <code>/api/scans</code> and <code>/api/stats</code> even with <code>PERSIST_SCANS=false</code> . This is the error encountered during setup.
Ports	Vite + API hard-pinned to 3000/3001 with <code>strictPort: true</code> and matching error strings. Conflict-prone on dev machines running other Node apps. Hardcoding the proxy target to a sibling port is brittle — env-driven <code>VITE_DEV_API_PORT</code> would be cleaner.
Tooling	No tests, no linter/Prettier config, no CI, no Dockerfile, no root README/CLAUDE.md. <code>.gitignore</code> is 2 lines.
Secrets	<code>.env</code> is gitignored. Service-role key only used server-side — correct.

2. Backend implementation — 7/10

- Engine layer (`query.js`) is the strongest part.** All four providers return a uniform `{ text, sources, error? }`, and `server.js` wires them via a single `ENGINE_FNS` map (`server.js:38–43`). Adding a 5th provider is a 1-function change.
- `prompts.js` does atomic-rename JSON persistence with length + `{{brand}}` validation — well done.
- `save.js:11–21` validates `scan_id` with a helpful error. Good defensive boundary.
- `server.js` is monolithic (456 LOC).** Routes, in-memory aggregation in `/api/stats` (lines 137–305), brand normalization, and ignored-brand env logic all share one file. The stats handler re-iterates rows 3x and is fine at <1k rows but won't scale — move aggregation to SQL once persistence is on.
- `/api/scan` is fully sequential** (`server.js:350–408`): `brands × prompts × engines` nested for `await`. $4 \times 5 \times 4 = 80$ serial LLM calls, which is why `requestTimeout` is set to 15 min. `Promise.all` across engines per prompt would cut this 4x.
- No input validation library (Zod/Joi), no rate limiting, no auth, CORS is wide open (`app.use(cors())`). Acceptable for local/demo, not for any public deploy.
- `analyze.js` is heuristic keyword matching — easily gamed (no negation: “not great” → positive). Fine for v0, but flag it: numbers in the dashboard are weakly grounded.

3. Frontend implementation — 5/10

- **frontend/src/app.js is 1,668 lines in a single function (mount())**. All HTML templates, event wiring, charts, localStorage cache, and stats recomputation live in one closure. No componentization; no framework. The top ~330 lines of pure helpers (escape, format, brand resolution, chart renderers) are clean and reusable; the bottom 1,300 are not.
- Charts are hand-rolled SVG/divs — keeps bundle tiny but ties every data shape to a render path. Hard to extend.
- **Duplicated stats logic**: server computes a stats blob in `/api/stats` ; the client recomputes a similar blob in `statsFromAudit` (`app.js:1065`) for the localStorage-only path. Two sources of truth for the same metrics.
- `apiUrl()` + `VITE_API_URL` story is well-thought-out: dev uses Vite proxy, prod injects build-time origin.
- `fetchJson` (`app.js:18-50`) handles HTML-instead-of-JSON cases nicely, but its error strings still mention `port 3000/3001` — stale once ports change.
- XSS posture: `escapeHtml` is used consistently inside the manual `innerHTML` strings. Reasonable.

4. Code reusability

Layer	Reusable?
Backend engine adapters	✓ Excellent — uniform contract, single registry
Prompt templates	✓ Substitution + persistence is a clean module
Frontend pure helpers (top of <code>app.js</code>)	✓ Could be lifted into <code>utils/</code> files as-is
Frontend views / charts	✗ Tangled in <code>mount()</code> ; would need extraction before reuse
Stats computation	✗ Duplicated client/server, no shared schema
Config / logging / error envelope	✗ No abstraction; <code>process.env.X</code> and <code>console.*</code> scattered

5. Top fixes, ranked by ROI

1. **Add `create-scans.sql` base migration** — unblocks fresh setups (drafted at `backend/supabase-migrations/create-scans.sql` , awaiting apply).
2. **Fix the `.env` path** in `server.js:2` and `supabaseClient.js:4` to read `__dirname + "/.env"` , or move the file to root. Pick one canonical location.
3. **Parallelize `/api/scan`** across engines per prompt — biggest UX win, ~4× faster runs.
4. **Move `/api/stats` aggregation to SQL** before turning on `PERSIST_SCANS` at any real volume.
5. **Split `app.js`** along view boundaries (home / test-results / run-audit / prompts) — even without a framework, four files of ~400 lines each is dramatically more navigable.
6. **Add a README** with env-file location, port assumptions, and migration order. The current setup story is reverse-engineered from comments inside `vite.config.js` and `netlify.toml` .

TL;DR: Tasteful v0 with one excellent abstraction (engine layer), two real setup bugs (missing base migration, env path mismatch), and one chunk of structural debt (the 1.7k-line frontend file). Worth shipping after fixes 1 + 2; worth refactoring after fix 5.